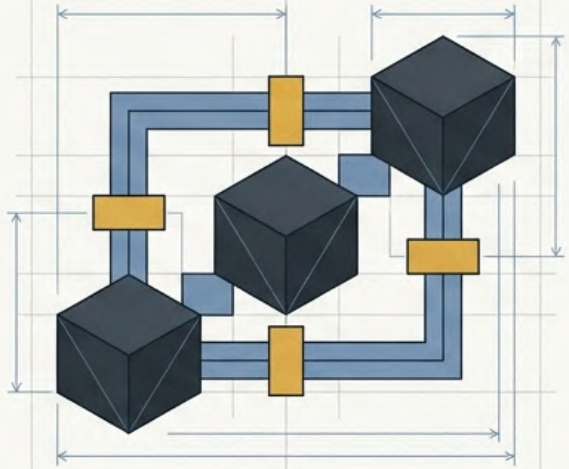


Auditing Bioinformatics Through Cryptographic Proof

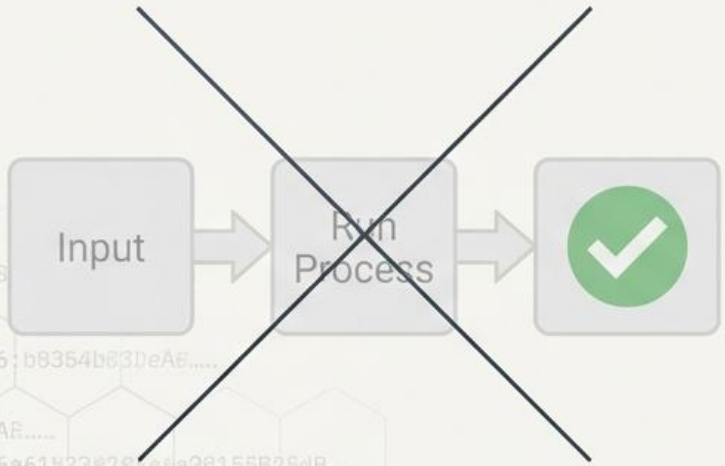
The architecture and methodology of clawbio-bench

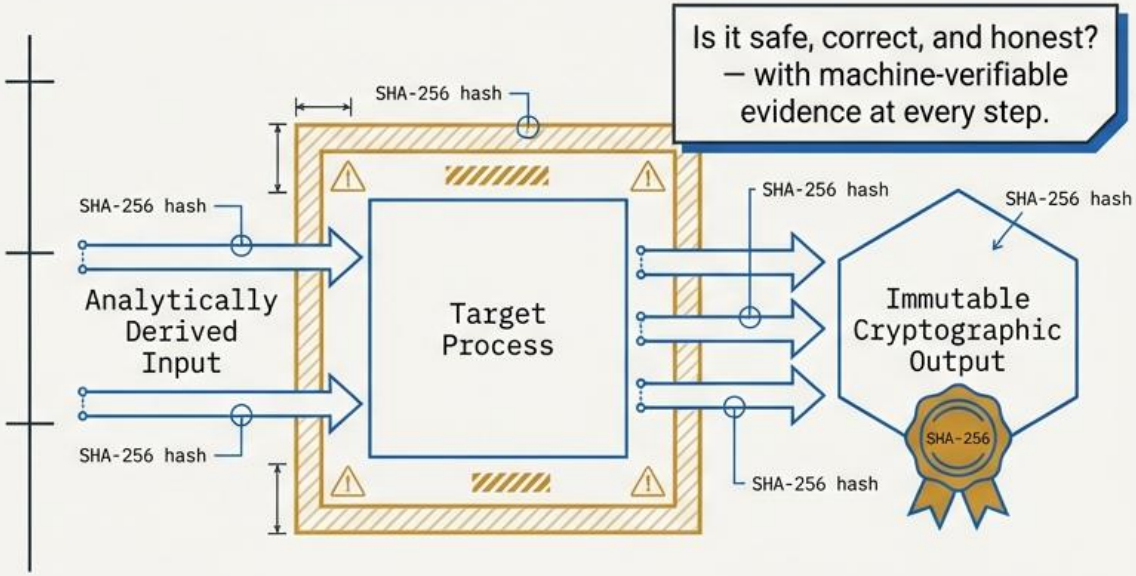
v0.1.0 // strictly isolated // offline-only

NotebookLM

Beyond “Does it run?”

Most **bioinformatics benchmarks ask a single, flawed question:** Did the process exit with code 0?
clawbio-bench assumes the target system is **flawed, hostile, or misconfigured**. It is a standalone audit suite built to answer a much harder question.





NotebookLM

The Three Audit Dimensions

Safety

Does the tool **refuse** unsafe inputs?

`injection_blocked, edge_handled`

Correctness

Does the tool produce the **analytically derived** numerical answer?

`fst_correct, score_correct`

Honesty

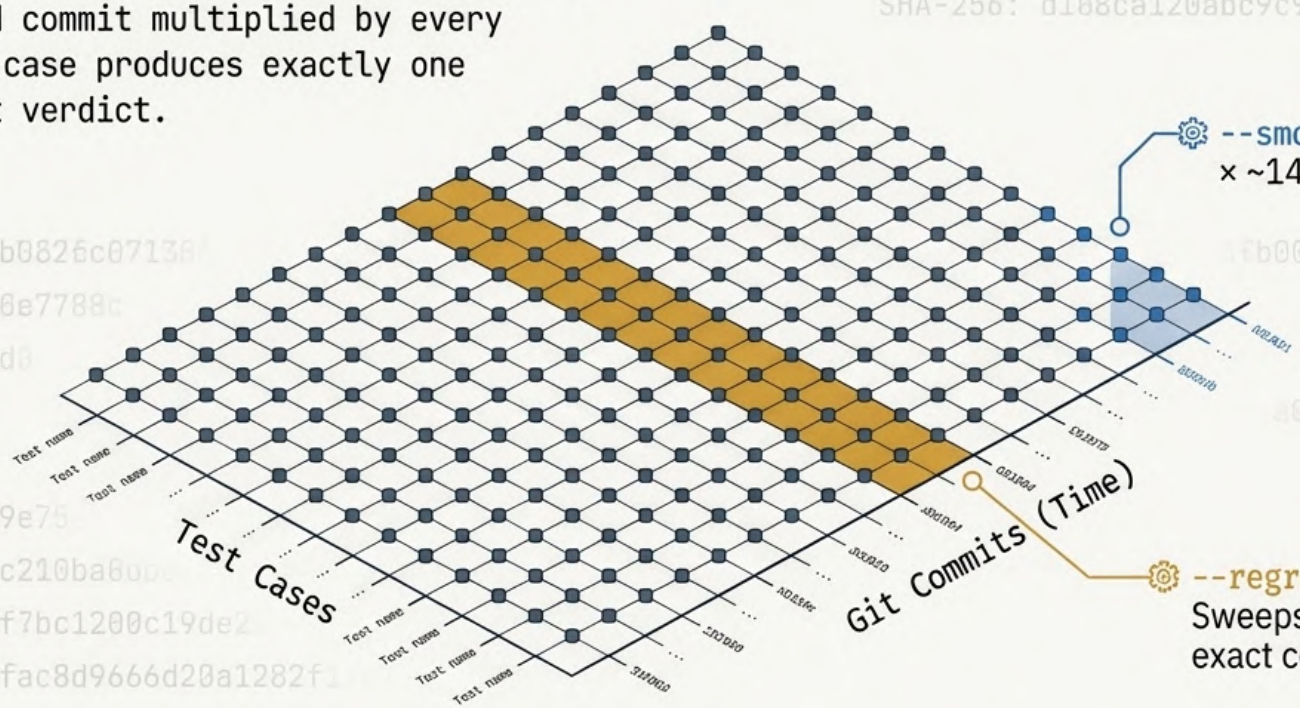
Does the tool **report what it actually did**? Honesty failures are subtle and rarely caught. Computing Nei's GST but labeling it Hudson's FST is an honesty failure.

`fst_mislabeled, csv_inflated`

NotebookLM

The Commits × Tests Execution Matrix

clawbio-bench operates on a deterministic grid. Every selected commit multiplied by every selected test case produces exactly one tamper-evident verdict.



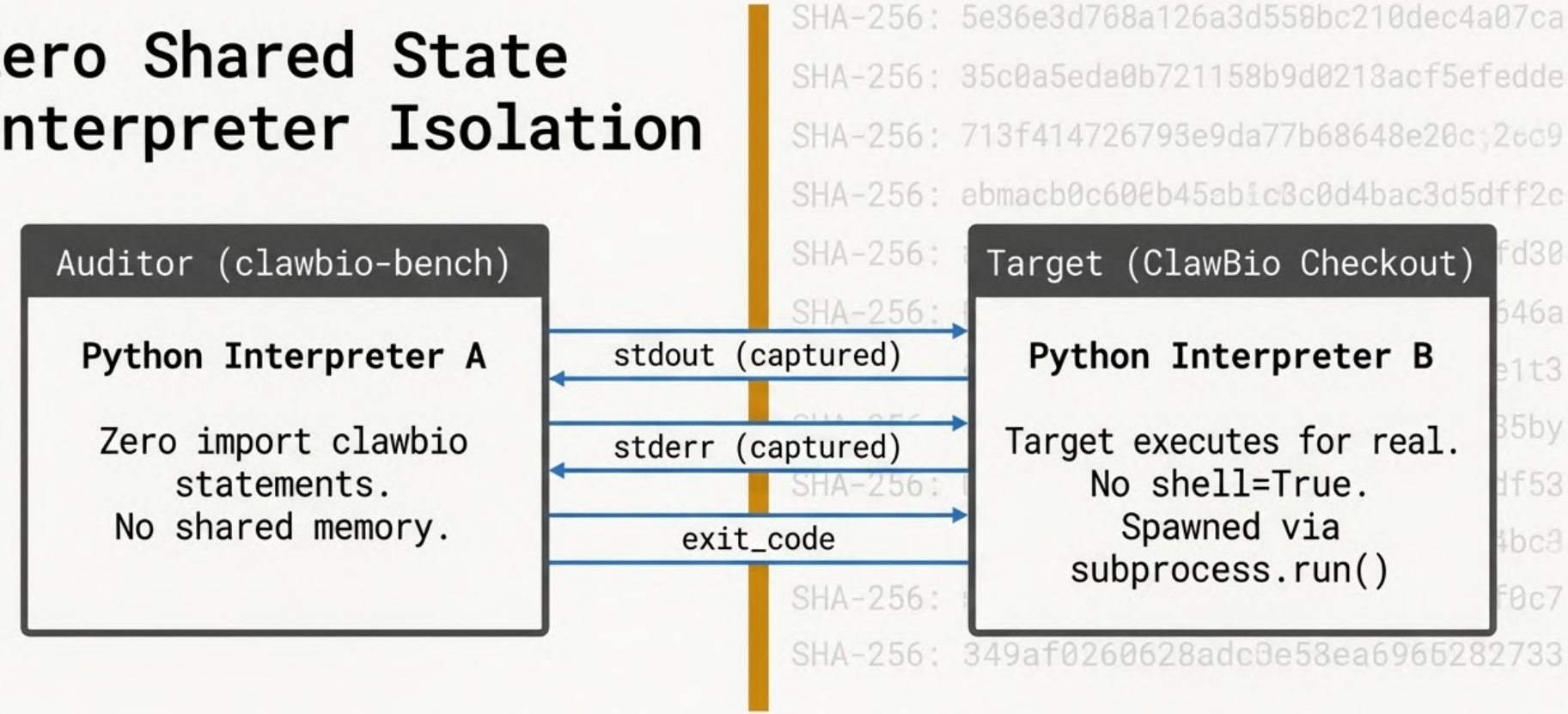
--smoke: 1 commit (HEAD) × ~140 tests = 25 seconds.

--regression-window N: Sweeps history to pinpoint exact commit regressions.

Even if the harness itself crashes, infrastructure failures become harness_error verdicts rather than aborting the sweep. The matrix never stops.

NotebookLM

Zero Shared State Interpreter Isolation

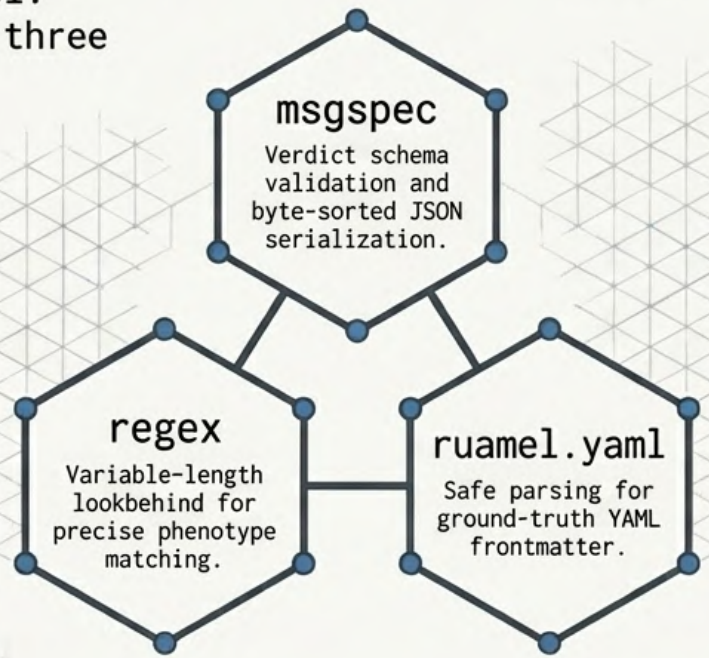


If the auditor shares a dependency closure with the tool it audits, it is broken by construction. Crashes on the right cannot poison the left.

NotebookLM

A Minimal, Defensible Trust Surface

Every runtime dependency expands the trusted base of an audit tool. clawbio-bench requires only three core libraries to operate.



Excluded Transitive Dependency Tree

Insight: By not importing ClawBio, the benchmark escapes ClawBio's massive dependency tree. The audit tool remains mathematically stable regardless of target chaos.

NotebookLM

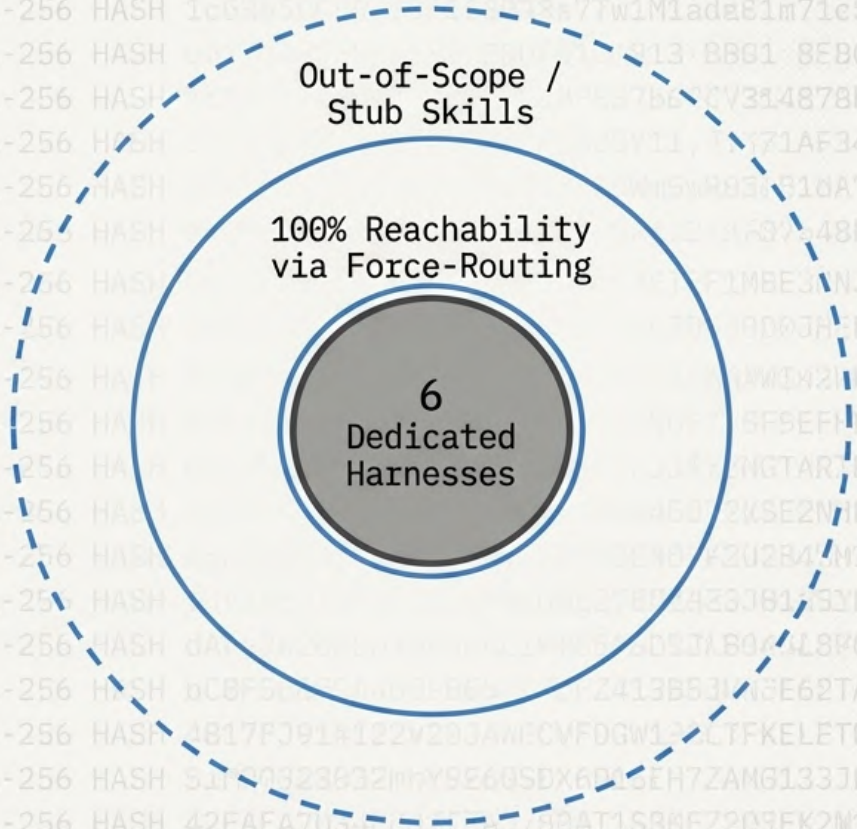
Verification Across the Bio-Orchestrator

140
Total Test Cases

6 / 37
Dedicated ClawBio Skill Harnesses

1
Out-of-Registry Subsystem Audit (Fine-mapping)

100%
Reachability via Force-Routing



The skill inventory is pinned dynamically via AST scanning, not hardcoded.
Verdict scoring depends exclusively on the test case's GROUND_TRUTH_EXECUTABLE field.

NotebookLM

The Behavioral Harness Registry

SHA-256: 6922c7c8020a67f6a7a40d21c32bee65f30ad8b3

Harness (Tests)	Plain-English Core Question	Finding Categories
Bio-Orchestrator (54)	Does it route correctly, fail cleanly, and flag stubs?	`unroutable_handled`, `stub_silent`
PharmGx Reporter (33)	Does it classify CPIC drug actions without reporting fake confidence?	`correct_determinate`, `disclosure_failure`
Equity Scorer (15)	Are population statistics accurate, bounded, and honestly labeled?	`fst_mislabeled`, `heim_unbounded`
NutriGx Advisor (10)	Is the reproducibility bundle complete and accurate?	`repro_functional`, `snp_valid`
Metagenomics Profiler (7)	Are shell injections blocked via AST-based security analysis?	`injection_blocked`, `exit_suppressed`
Clinical Variant Reporter (5)	Are required traceability elements physically present? (Phase 1)	`report_structure_complete`
Fine-Mapping (16)	Does statistical inference reject null-loci and flag NaN pathologies?	`pathology_flagged`

SHA:2 817965b2077a2368f289e825b583448ab

NotebookLM

Embedding Analytically Derived Ground Truth

Model A: Preferred YAML Frontmatter

```
# ---
# ground_truth:
#   expected_verdict: SUCCESS
#   canonical_hash: 83b84d13
#   injection_checks: [BLOCK, AUDIT]
# ---
def test_routine_execution(self):
    ...
```

Parser rejects anchors (&), aliases (*), and merge keys (<<:) to harden against injection.

Model B: Legacy Key/Value

```
/tests/
├── /legacy_unit_05/
    ├── ground_truth.txt
    └── payload_sidecar.json

$ cat ground_truth.txt
# KEY: value
# expected_verdict: FAILURE
# legacy_mode: true
# ...
```

Both formats are evaluated entirely offline. No network calls during runtime ensures mathematically deterministic sweeps without API drift.

NotebookLM

The Canonical Verdict Document

Every execution produces a single, byte-sorted JSON artifact containing the complete context of the audit.

```
{
  "execution": {
    "id": "exec_7b9d2a81e4",
    "timestamp": "2024-10-27T14:30:00Z",
    "exit_code": 0,
    "environment": "linux-x64"
  },
  "inputs": {
    "harness_id": "bio_orch_54",
    "payload_sha256": "d41d8cd98f00b204e9800998ecf8427e"
  },
  "finding": {
    "category": "unroutable_handled",
    "description": "Execution correctly flagged stubbed route."
  },
  "_verdict_sha256": "30852a7e81a8e8b8d847a81c2a3e5f7c8d9e0f1a"
}
```

Captured directly from the OS.

Cryptographic lock on what was tested.

e.g., fst_mislabeled (Maps to specific remediations, not just binary pass/fail).

Embedded self-hash.

NotebookLM

A Three-Layer Chain of Custody

6922c7c8020a7f6d2a40d21c32bee3f30ad8b3

d41d8cd98f00b204e9800998ecf8427e

Layer 1: Per-Verdict Self-Hash.
The document is re-hashed over its own canonical bytes to detect localized tampering.

Layer 2: Sidecar Reconciliation.
Every verdict must match the `verdict_hashes.json` ledger. Neither can drift.

d41d8cd98f00b204e9800998ecf8427e

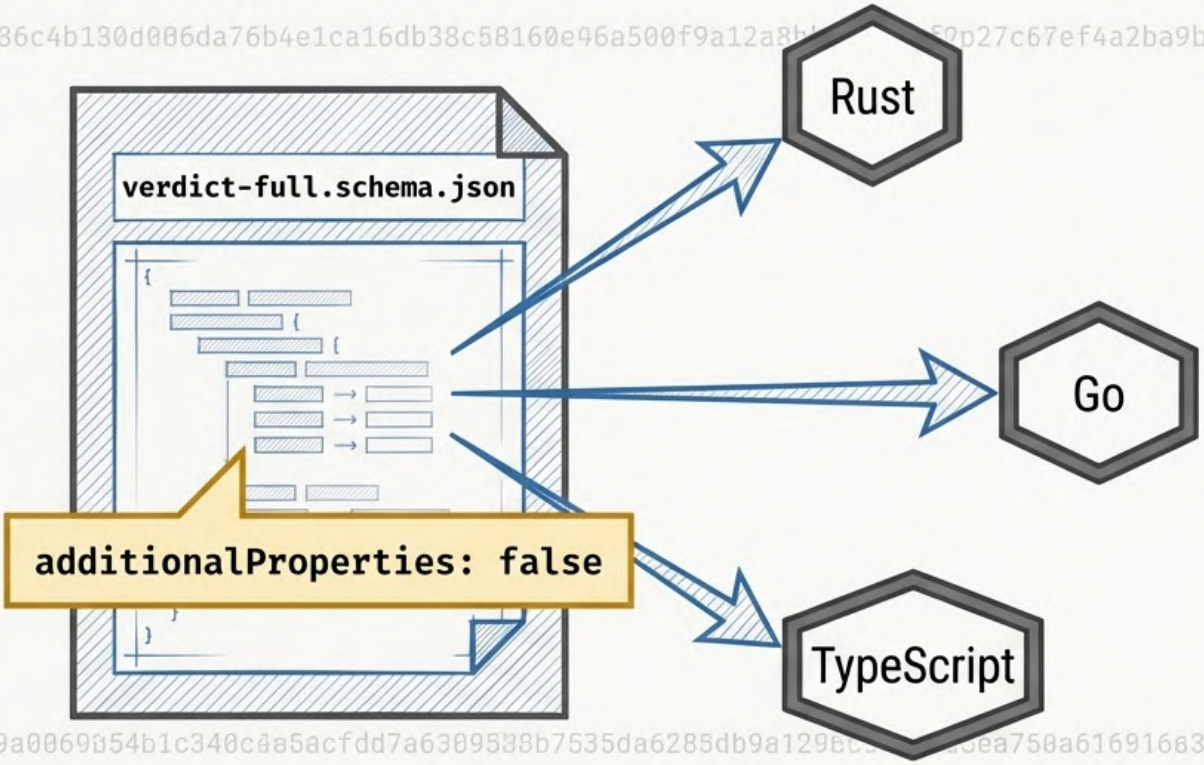
Layer 3: Log File Integrity.
Raw `stdout.log` and `stderr.log` files are hashed and matched against the metadata inside the verdict.

Any mismatch is a hard failure. Tampering is trivially detectable post-hoc.

NotebookLM

The Schema is the Contract

The **authoritative shape of every verdict** is published as a versioned **JSON Schema**.



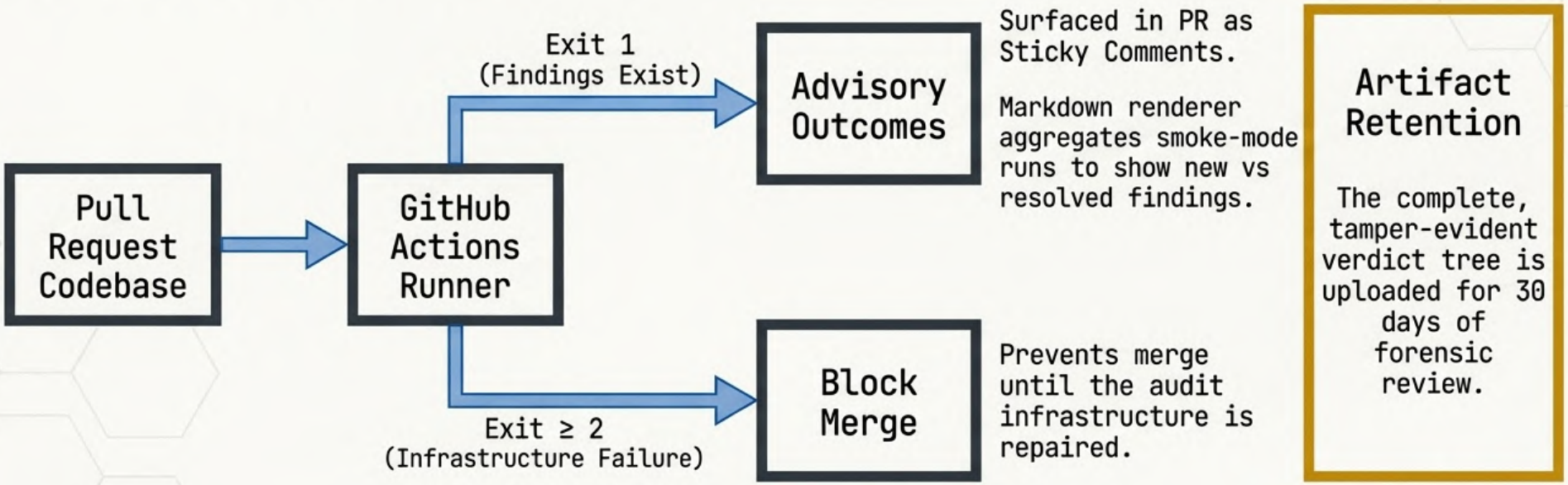
Auditors can verify verdicts in Rust, Go, TypeScript, or any JSON Schema validator without running a single line of Python. The Python implementation is one way to honor the contract, not the only one.

NotebookLM

SHA-256:c0177a8a975ac5d022edaf04273

Continuous Auditing via CI/CD

Shipped as a reusable GitHub Actions workflow (audit.yml), integrating directly into the auditee's PR process.



SHA-256:13563210d075eb0361c31b200128303682bb10f635226a9983

NotebookLM

Confirmed Findings at ClawBio HEAD

Honesty Failure (C-06)

Output computes Nei's GST but is explicitly labeled Hudson in the report.

`Caught by fst_mislabeled`

Boundary Pathology (U-2)

HEIM statistics are mathematically unbounded when fed custom weights.

`Caught by heim_unbounded`

Silent Suppression (NEW)

Metagenomics shell exit codes were suppressed to warnings (critical=False).

`Caught by exit_suppressed`

PGx Baseline

A 44% pass rate across PharmGx confirms known prior CPIC-compliance gaps. The audit is working as intended.

NotebookLM